



算法分析与设计

Analysis and Design of Algorithm

Lecture 02



要点回顾

■ 基础知识

- 算法、数据结构和程序的概念及其区别与联系
- 算法的作用和描述方法

■ 算法设计

- 举了两个例子说明算法设计的一般过程
 - 抽象为数学模型
 - 根据模型特点发掘规律
 - 写出具体求解过程

■ 算法分析

- 基于经验的评估方法（排序算法的实验比较）
- 基于理论的评估方法（平均和最坏情况分析）



思考

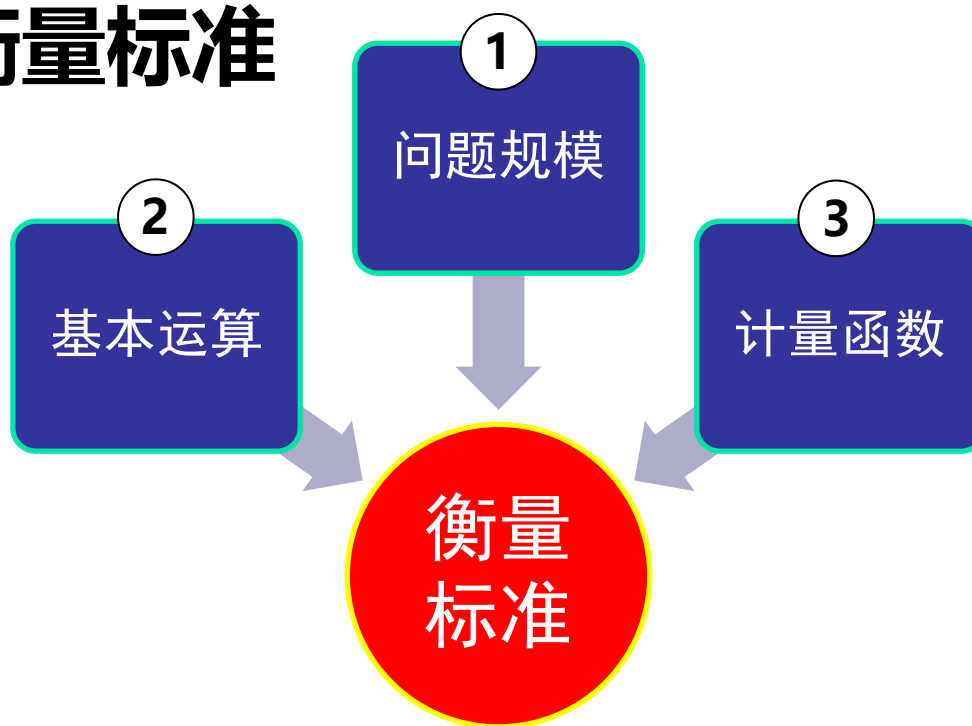
- 如何改进之前的顺序检索算法?
- 并分析其最坏情况和平均情况的时间复杂度

顺序检索算法改进:

- $j=1$, 将 x 与 $L[j]$ 比较,
 - 如果 $x=L[j]$, 则算法停止, 输出 j ;
 - 如果 $x > L[j]$, 则把 j 加1, 继续比较; 如果 $x < L[j]$, 则停机并输出0;
 - 如果 $j > n$, 则停机并输出0.

算法好坏的衡量尺度

- 用所需的计算时间来衡量一个算法的好坏，不同的机器相互之间无法比较。
 - 能否有一个独立于具体计算机的客观衡量标准。
- 常见的衡量标准





1、问题规模

- **问题的规模：**一个或多个整数，作为输入数据量的测度。
 - **数表的长度**(数据项的个数)，(问题：在一个长度为 n 的数组中寻找元素 x)；
 - **矩阵的最大维数**(阶数) (问题：求两个实矩阵相乘的结果)
- **输入规模通常用 n 来表示，也可有两个以上的参数**
 - 图中的顶点数和边数(图论中的问题)

2、基本运算

- 概念：
 - 也称为“元运算”
 - 指执行时间可以被一个常数限定，只与**环境**有关。
- 因此，分析时只需要关心执行的基本运算次数，而不是它们执行确切时间。
 - 例子：

$$\left. \begin{array}{l} a \text{ 次加法, 一个加} \leq t_+; \\ m \text{ 次乘法, 一个乘} \leq t_\times; \\ s \text{ 次赋值, 一个赋值} \leq t_\leftarrow; \end{array} \right\} \quad t \leq at_+ + mt_\times + st_\leftarrow$$

只和**基本运算**相关

机器、语言编译





2、基本运算

- 一般可以认为加法和乘法都是一个单位开销的运算。
 - 理论上，这些运算都不是基本运算，因为操作数的长度影响了执行时间。
- 实际，只要实例中操作数长度相同，即可认为是基本运算。



2、基本运算

■ 例如

- 在一个表中寻找数据元素 x ： x 与表中的一个项进行比较；
- 两个实矩阵的乘法：实数的乘法（及加法）
 $C=AB$ ；
- 将一个数表进行排序，表中的两个数据项进行比较。

■ 通常情况下，讨论一个算法优劣时，我们只讨论基本运算的执行次数。

因为它是占支配地位的，其他运算可以忽略。



3、计量函数

- 用输入规模的某个函数来表示算法的基本运算量，称为**算法的时间复杂性(度)**。

用 $T(n)$ 或 $T(n, m)$ 来表示，例如：

- $T(n)=5n$
- $T(n)=3n\log n$
- $T(n)=4n^3$
- $T(n)=2^n$
- $T(n, m)=2(n + m)$

怎样提高效率

- 硬件速度增加，算法效率还那么重要吗？
 - 大小为 n 的实例的执行，需要 $10^{-4} \times 2^n$ 秒。则：

$$n = 10 \quad 10^{-4} \times 2^{10}$$

$$n = 20 \quad 10^{-4} \times 2^{20} \quad \text{扩大} 2^{10} \text{倍}$$

$$n = 30 \quad 10^{-4} \times 2^{30} \quad \text{扩大} 2^{10} \times 2^{10} \text{倍}$$

-
- **假设**计算机性能**提高100倍**，则需要 $10^{-6} \times 2^n$ 秒。
一年时间还是求不出 $n=45$ 的实例
即，新的计算机最多解决 $n+\log_2 100$ 的实例， $n+7$ 。



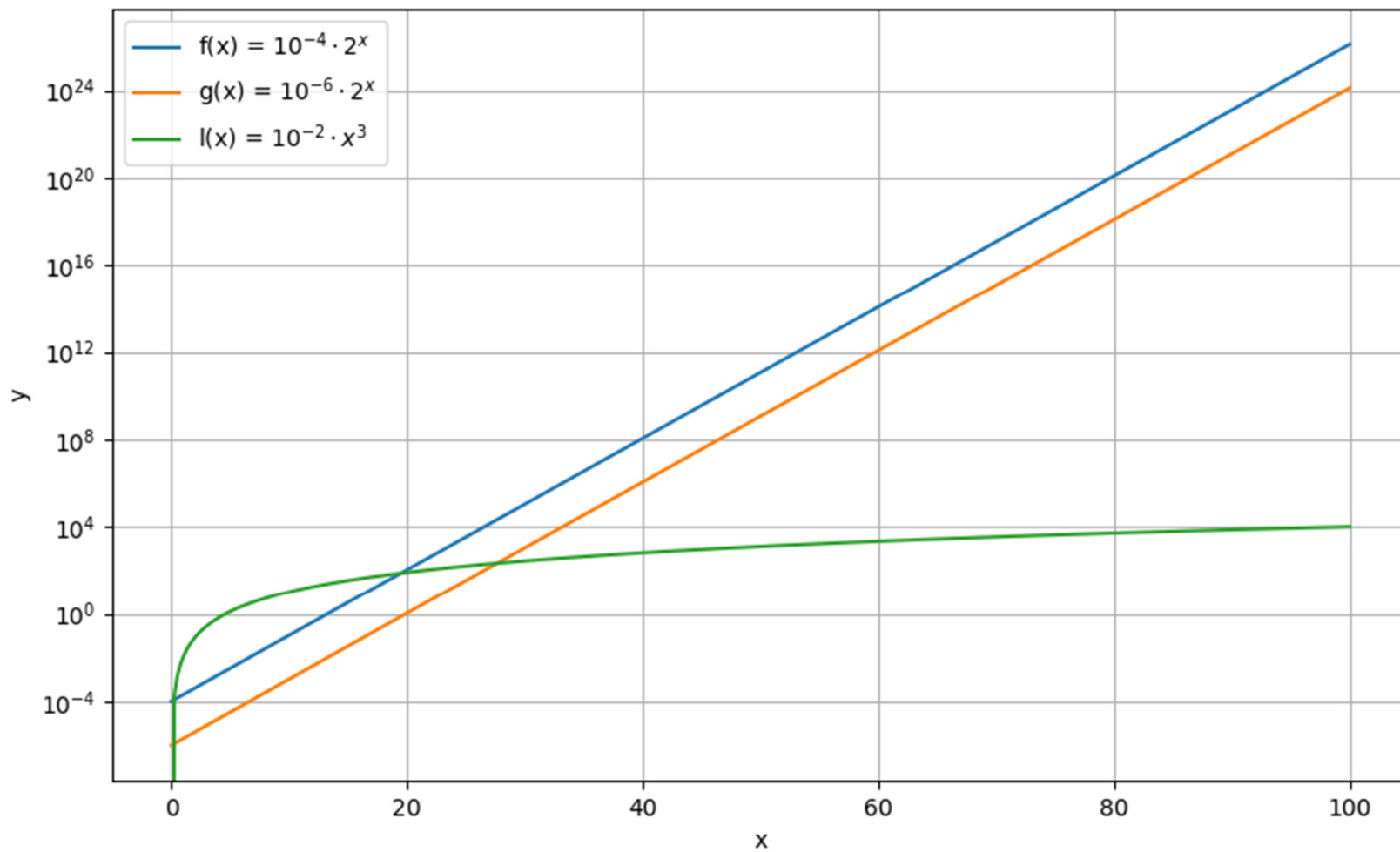
怎样提高效率

- **假设**改进算法，在原来计算机上，需 $10^{-2} \times n^3$ 秒。

$n = 10$	$\approx 10s$
$n = 20$	$1 \sim 2 \text{ min}$
$n = 30$	$\leq 4.5 \text{ min}$

因此，用一天可以解决大小超过200的实例，一年处理 $n \approx 1500$ 实例。

怎样提高效率





怎样提高效率

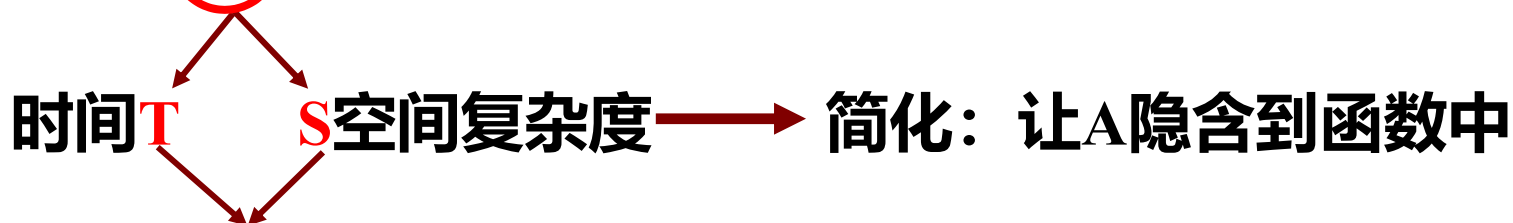
	n	$n \log_2 n$	n^2	n^3	1.5^n	2^n	$n!$
$n = 10$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	4 sec
$n = 30$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	18 min	10^{25} years
$n = 50$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	11 min	36 years	very long
$n = 100$	< 1 sec	< 1 sec	< 1 sec	1 sec	12,892 years	10^{17} years	very long
$n = 1,000$	< 1 sec	< 1 sec	1 sec	18 min	very long	very long	very long
$n = 10,000$	< 1 sec	< 1 sec	2 min	12 days	very long	very long	very long
$n = 100,000$	< 1 sec	2 sec	3 hours	32 years	very long	very long	very long
$n = 1,000,000$	1 sec	20 sec	12 days	31,710 years	very long	very long	very long

数据来源: Princeton University本科算法兴趣小组单机实验

深深地印在脑海里!

算法的复杂性

- 算法的复杂性是算法运行所需计算机资源的量。
 - 需要时间的→时间复杂性 (Time Complexity)
 - 需要空间的→空间复杂性 (Space Complexity)
- 反映算法的效率，并与运算计算机独立。
- 依赖于问题规模 N ，输入 I 和算法本身 A ，表示为 $C=F(N, I, A)$ 是一个三元函数。

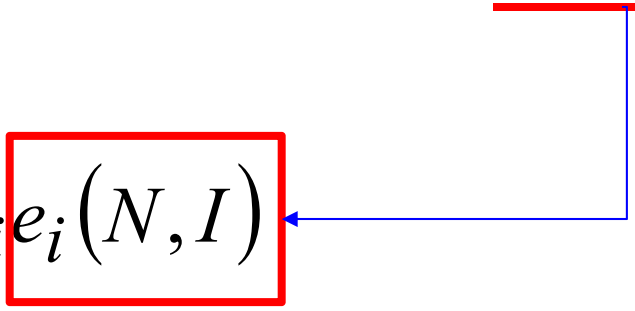


两者类似，但S简单

所以通常研究 $T(N, I)$ 在一台抽象计算机上运行所需时间。

T(N, I)的概念

- 设抽象计算机的元运算有 k 种，记为 $O_1, \dots, O_k$ ，每执行一次所需时间为 $t_1, \dots, t_k$ 。
- 对算法A，用到元运算 O_i 的次数为 e_i 与 N, I 相关。

$$T(N, I) = \sum_{i=1}^k t_i e_i(N, I)$$


T(N, I)的概念

- 不可能规模 N 的每种合法输入 I 都去统计 $e_i(N, I)$, 对于 I 分别考虑:

最坏情况、最好情况、平均情况

$$T_{\max}(N) = \max_{I \in D_N} T(N, I) = \max_{I \in D_N} \sum_{i=1}^k t_i e_i(N, I) = \sum_{i=1}^k t_i e_i(N, I^*) = T(N, I^*)$$

$$T_{\min}(N) = \min_{I \in D_N} T(N, I) = \min_{I \in D_N} \sum_{i=1}^k t_i e_i(N, I) = \sum_{i=1}^k t_i e_i(N, \tilde{I}) = T(N, \tilde{I})$$

$$T_{\text{avg}}(N) = \sum_{I \in D_N} P(I) T(N, I) = \sum_{I \in D_N} P(I) \sum_{i=1}^k t_i e_i(N, I)$$

合法输入集



复杂性渐进性态

- 设 $T(N)$ 是前面定义的算法 A 复杂性函数。
 - N 递增到无限大, $T(N)$ 递增到无限大
 - 如存在 $\tilde{T}(N)$, 使 $N \rightarrow \infty$ 时, 有 $\frac{T(N) - \tilde{T}(N)}{T(N)} \rightarrow 0$
称 $\tilde{T}(N)$ 是 $T(N)$ 当 $N \rightarrow \infty$ 的渐进性态。
- 在数学上, $\tilde{T}(N)$ 是 $T(N)$ 当 $N \rightarrow \infty$ 的渐进表达式, 通常 $\tilde{T}(N)$ 是 $T(N)$ 中略去低阶项所留下的主项。
 - $\tilde{T}(N)$ 比 $T(N)$ 简单。



复杂性渐进性态

- 例如： $T(N) = 3N^2 + 4N \log N + 7$

$$\tilde{T}(N) = 3N^2$$

- 由 $\frac{T(N) - \tilde{T}(N)}{T(N)} = \frac{4N \log N + 7}{3N^2 + 4N \log N + 7} \rightarrow 0$

- 因为 $N \rightarrow \infty$, $T(N) \rightarrow \tilde{T}(N)$

- 所以有理由用 $\tilde{T}(N)$ 来替代 $T(N)$ 来度量A。



衡量算法的效率

- 当比较两个算法的渐近复杂性的阶不同时，只要确定各自的阶，即可判定哪个算法效率高。

等价于

- 只要关心 $\tilde{T}(N)$ 的阶即可，不必考虑其中常数因子。
- 简化算法复杂性分析的方法和步骤，只要考察问题的规模充分大时，算法复杂性在渐近意义下的阶。

回顾一下排序算法性能比较

类别	排序方法	时间复杂度			空间复杂度	稳定性
		平均时间	最好时间	最坏时间	辅助存储	
插入排序	直接插入	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	稳定
	希尔排序	$O(n \log n)$	$O(n)$	$O(n^s)$	$O(1)$	不稳定
选择排序	简单选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
	堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳定
交换排序	冒泡排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	稳定
	快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	不稳定
归并排序		$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定
基数排序		$O(d(r + n))$	$O(d(r + n))$	$O(d(r + n))$	$O(rd + n)$	稳定
注：希尔排序 s 是所选分组($1 < s < 2$);基数排序中, r 和 d 分别为关键字的基数和长度						



渐近分析的记号

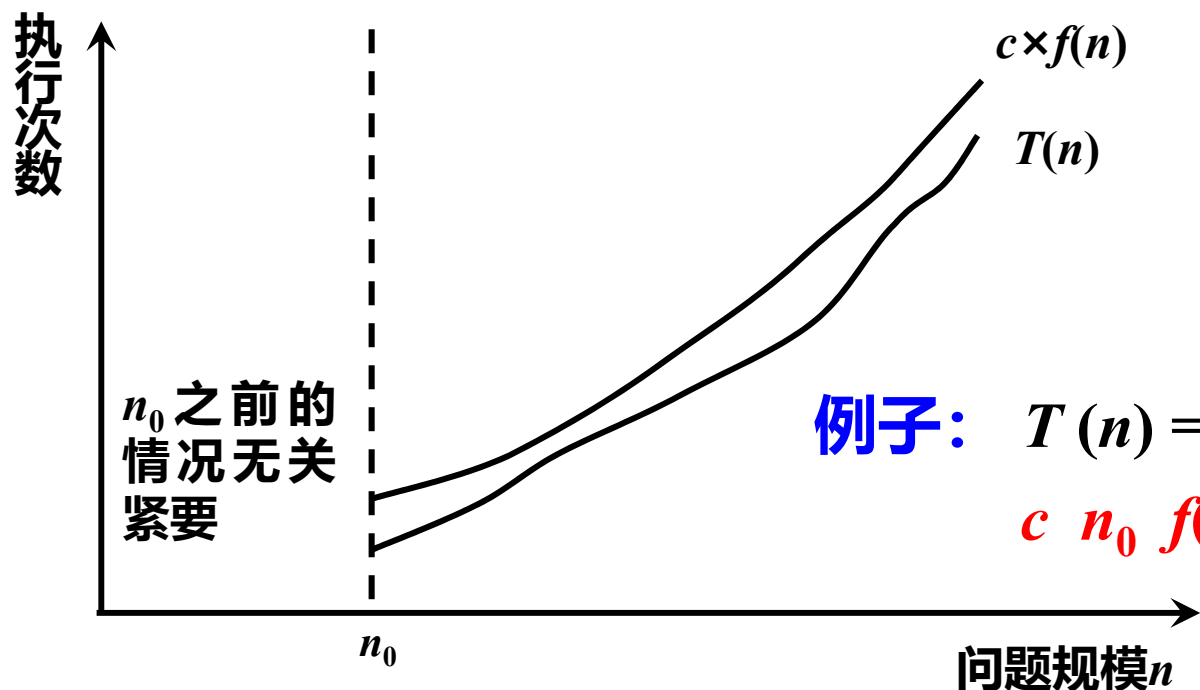
- 渐近上界记号 O
- 渐近下界记号 Ω
- 紧渐近界记号 Θ
- 非紧上界记号 o
- 非紧下界记号 ω

下面的讨论中，对所有 n ， $f(n) \geq 0$ ， $T(n) \geq 0$

渐近分析的记号

■ 渐近上界记号 O

- 若存在两个正的常数 c 和 n_0 ，使得对所有 $n \geq n_0$ ，都有： $T(n) \leq c \times f(n)$ ，则称 $T(n) = O(f(n))$

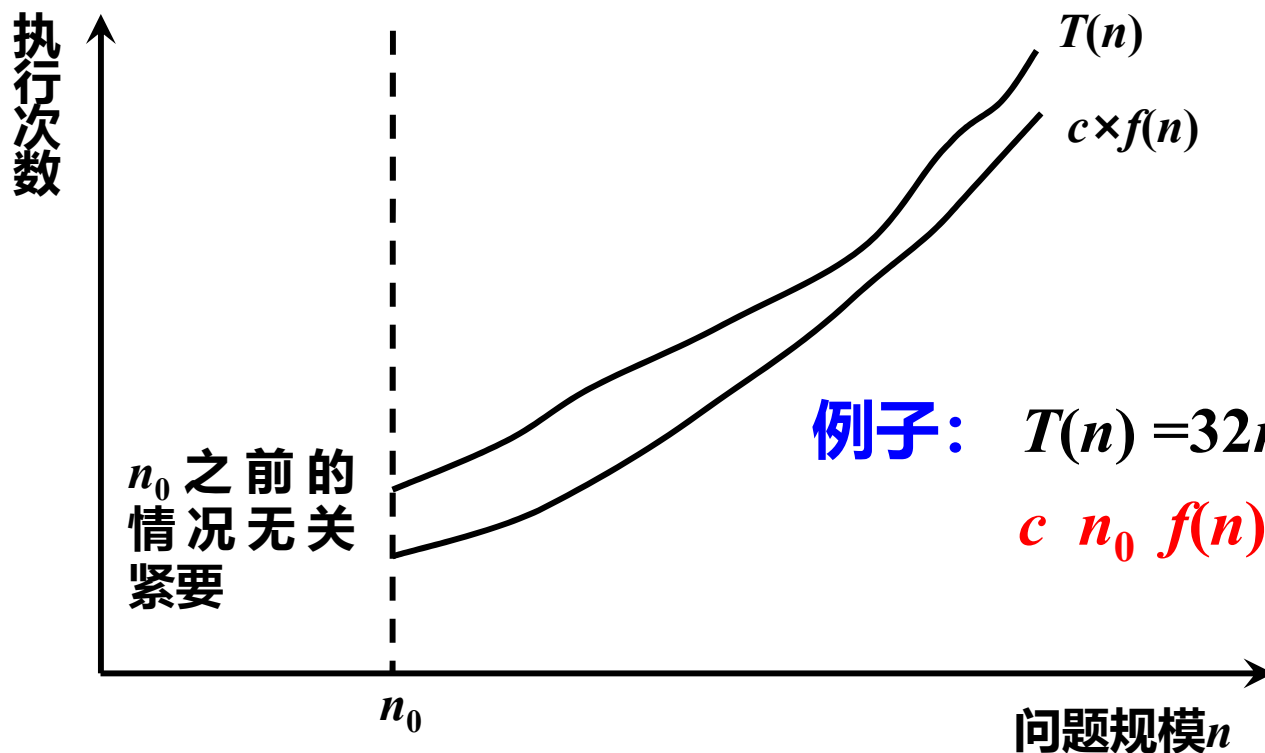


例子： $T(n) = 32n^2 + 17n + 1$
 $c \quad n_0 \quad f(n) = ?$

渐近分析的记号

■ 渐近下界记号 Ω

- 若存在两个正的常数 c 和 n_0 ，使得对所有 $n \geq n_0$ ，都有： $T(n) \geq c \times f(n)$ ，则称 $T(n) = \Omega(f(n))$

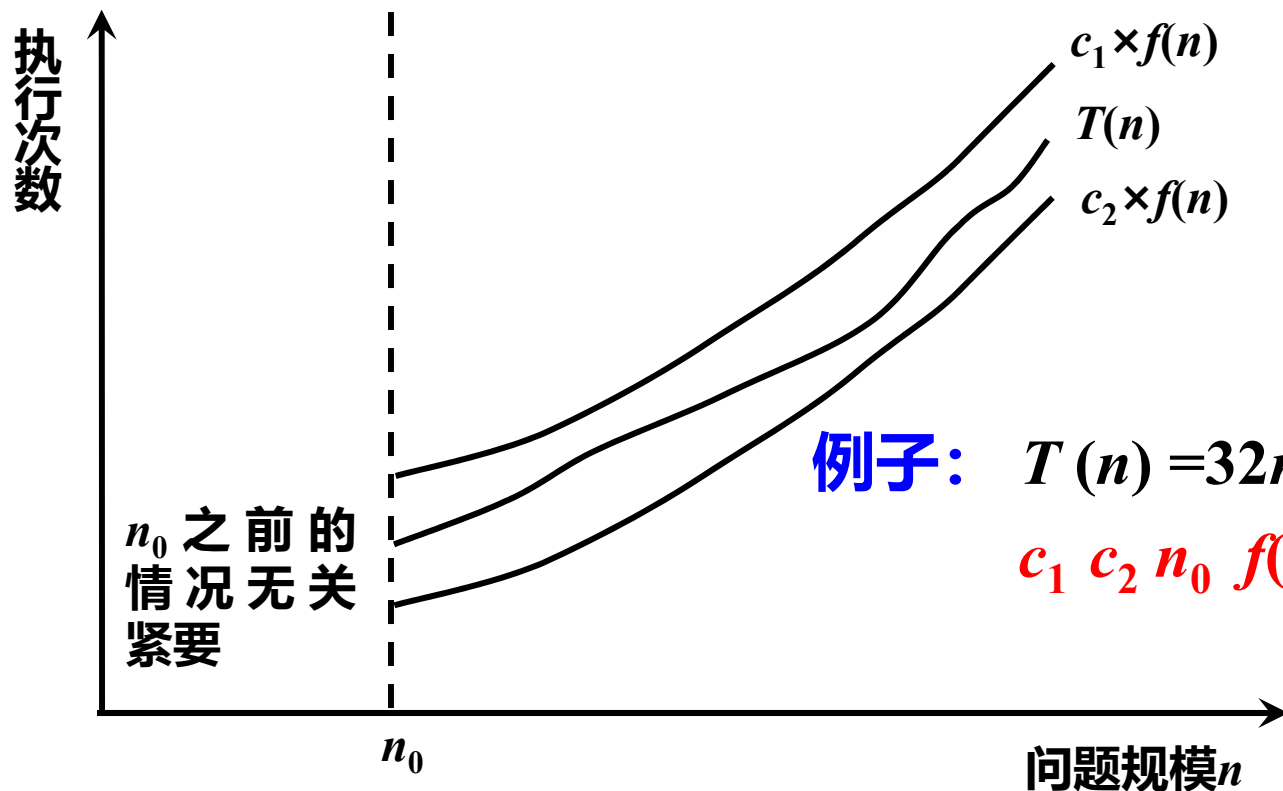


例子： $T(n) = 32n^2 + 17n + 1$
 $c \ n_0 \ f(n) = ?$

渐近分析的记号

■ 紧渐近界记号 Θ

- 若存在三个正的常数 c_1 、 c_2 和 n_0 ，使得对所有 $n \geq n_0$ ，都有： $c_1 \times f(n) \geq T(n) \geq c_2 \times f(n)$ ，则称 $T(n) = \Theta(f(n))$





渐近分析的记号

■ 非紧上界记号 o

- $o(f(n)) = \{ T(n) \mid \text{对于任何正常数 } c > 0, \text{ 存在正数 } n_0 > 0 \text{ 使得对所有 } n \geq n_0 \text{ 有: } 0 \leq T(n) < cf(n) \}$
- 等价于 $T(n) / f(n) \rightarrow 0$, as $n \rightarrow \infty$ 。

■ 非紧下界记号 ω

- $\omega(f(n)) = \{ T(n) \mid \text{对于任何正常数 } c > 0, \text{ 存在正数 } n_0 > 0 \text{ 使得对所有 } n \geq n_0 \text{ 有: } 0 \leq cf(n) < T(n) \}$
- 等价于 $T(n) / f(n) \rightarrow \infty$, as $n \rightarrow \infty$ 。
- $T(n) \in \omega(f(n)) \Leftrightarrow f(n) \in o(T(n))$

例：渐近意义下的 O

- 如果存在正的常数 C 和自然数 N_0 ，使得当 $N \geq N_0$ 时，有 $T(N) \leq C \times f(N)$ ，则称函数 $T(N)$ 当 N 充分大时上有界，且 $f(N)$ 是它的一个上界，记为 $T(N) = O(f(N))$ 。
- 也即 $T(N)$ 的阶不高于 $f(N)$ 的阶。
 - $\forall N \geq 1, 3N \leq 4N \Rightarrow 3N = O(N)$;
 - $\forall N \geq 1, N + 1024 \leq 1025N \Rightarrow N + 1024 = O(N)$;
 - $\forall N \geq 10,$
 $2N^2 + 11N - 10 \leq 3N^2 \Rightarrow 2N^2 + 11N - 10 = O(N^2)$;
 - $\forall N \geq 1, N^2 \leq N^3 \Rightarrow N^2 = O(N^3)$;
 - $N^3 \neq O(N^2)$, 无 $N \geq N_0$ 使得 $N^3 \leq CN^2 \quad N \leq C$;

反例



O 的运算性质

- $O(f) + O(g) = O(\max(f, g))$
- $O(f) + O(g) = O(f + g)$
- $O(f) \cdot O(g) = O(f \cdot g)$
- 如果 $g(N) = O(f(N)) \Rightarrow O(f) + O(g) = O(f)$
- $O(cf(N)) = O(f(N))$ 其中 c 是一个正的常数
- $f = O(f)$

下面考察性质的证明：



性质： $O(f) + O(g) = O(\max(f, g))$

证明： 设 $F(N) = O(f)$ 。根据符号 O 的定义，存在正常数 C_1 和自然数 N_1 ，使对所有 $N \geq N_1$ ，都有 $F(N) \leq C_1 f(N)$ 。

类似地， 设 $G(N) = O(g)$ ，则存在正的常数 C_2 和自然数 N_2 ， $N \geq N_2$ 有 $G(N) \leq C_2 g(N)$ 。



性质: $O(f) + O(g) = O(\max(f, g))$

$$\left. \begin{array}{l} \text{令 } C_3 = \max\{C_1, C_2\} \\ N_3 = \max\{N_1, N_2\} \\ h(N) = \max\{f, g\} \end{array} \right\} \begin{array}{l} \forall N > N_3 \\ F(N) \leq C_1 f(N) \leq C_1 h(N) \leq C_3 h(N) \end{array}$$

同理 $G(N) \leq C_2 g(N) \leq C_2 h(N) \leq C_3 h(N)$

所以 $O(f) + O(g) = F(N) + G(N) \leq C_3 h(N) + C_3 h(N)$
 $= 2C_3 h(N) = O(h) = O(\max(f, g))$

证毕!

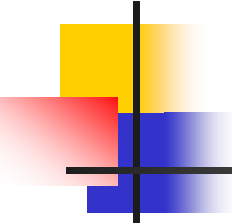


课堂练习

- 证明性质:

$$O(f) + O(g) = O(f + g)$$

$$O(f) \cdot O(g) = O(f \cdot g)$$



课程内容

NP完全性理论与近似算法

高级理论

随机化算法

线性规划与网络流

高级算法

**递归
分治**

**动态
规划**

**贪心
算法**

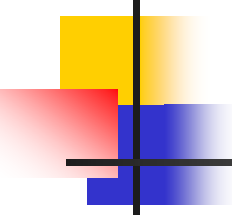
**回溯与
分支限界**

基础算法

算法分析与问题的计算复杂性

基础理论

NP完全性理论



重要的问题类型

- **查找/检索问题**

- 在给定的集合中寻找一个给定的值

- **排序问题**

- 按升序（降序）重新排列给定列表中的数据项

- **图问题**

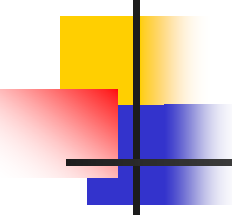
- 图的遍历、最短路径以及有向图的拓扑排序

- **组合问题**

- 寻找一个组合对象（排列或子集）满足一些重要特性

- **几何问题**

- 处理类似于点、线、多面体这样的几何对象



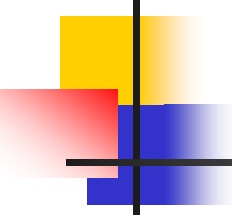
问题的复杂度

当为某一问题设计算法时，我们总是追求最好的复杂度。但是，怎样才能知道已达到最佳呢？我们必需考虑问题的复杂度。

- 问题的复杂度就是**任一个**解决该问题的算法所必需的运算次数。

例如，任何一个采用比较大小的办法将 n 个数排序的算法需要至少 $\lg n!$ 次比较才行。那么 $\lg n!$ 就是（基于比较的）排序问题的复杂度。因为没有一个算法可用少于 $\lg n!$ 次比较解决排序问题， $\lg n!$ 就成了算法复杂度的下界，即任一比较排序算法的复杂度必定为 $\Omega(\lg n!) = \Omega(n \lg n)$ 。

- 所以，如果能证明某个问题至少需要 $\Omega(f(n))$ 运算次数，那么 $\Omega(f(n))$ 就是所有解决该问题的算法的复杂度的下界。



问题的复杂度

- 反之，如果某一算法的复杂度是 $O(f(n))$ ，那么它解决的问题的复杂度不会超过 $O(f(n))$ 。
- 因此，任一算法的复杂度也是其所解决的问题的复杂度的上界。
- 通常在已知的某问题的复杂度下界和该问题最好算法的复杂度之间存在距离，算法研究的任务就是努力寻找更好的下界或更优的上界。

结论：找出问题的复杂度，即找出其算法的下界，是一重要的工作，因为它可以告诉我们是否还有改进当前算法的余地而省去很多徒劳无功的努力。

易解问题与难解问题

- 通常将存在**多项式时间**算法的问题看作是**易解问题** (Easy Problem) ;
 - 排序问题、查找问题、欧拉回路 → **问题规模**
- 而将需要**指数时间**算法解决的问题看作是**难解问题** (Hard Problem) 。
 - TSP问题、Hanio问题、Hamilton回路问题